

Compte Rendu — TP2 : Docker Compose : orchestrer le stack

Mastère Expert IT — CI/CD & DevSecOps

Groupe : gGODON-DAUVEL

Date : 08 juin 2026

Lien GitHub : [CI-CD_SemaineIntensive](#)

Table des matières

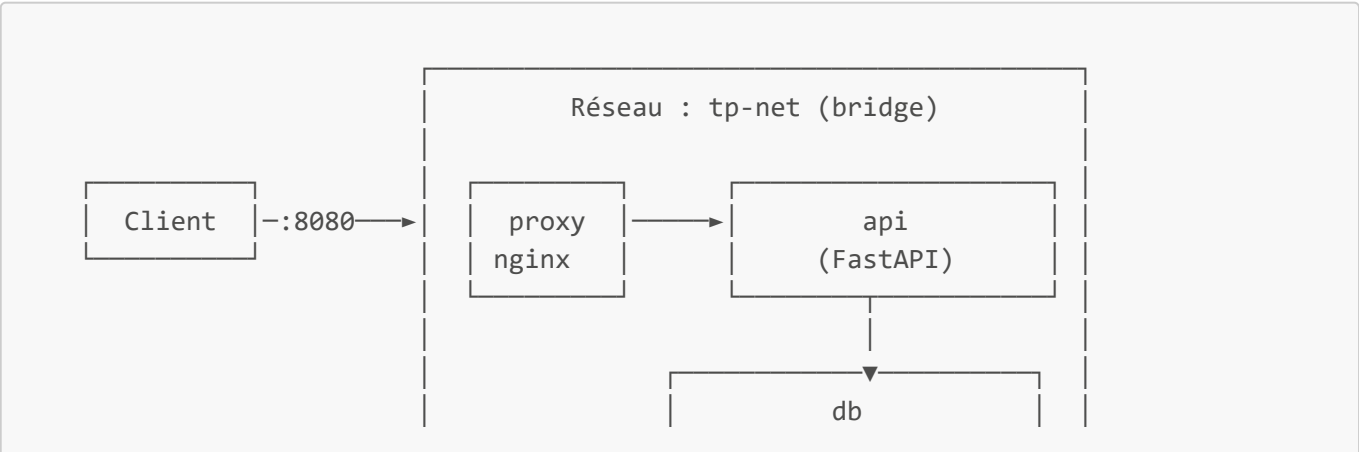
- 1. [Architecture du stack](#)
- 2. [Phase 1 — Squelette du stack](#)
- 3. [Phase 2 — Healthchecks et dépendances](#)
- 4. [Phase 3 — Configuration et secrets](#)
- 5. [Phase 4 — Overrides développement et production](#)
- 6. [Phase 5 — Limites de ressources et journalisation](#)
- 7. [Phase 6 — Vérification de bout en bout](#)
- 8. [Phase 7 — Résilience](#)
- 9. [Mini-questionnaire](#)

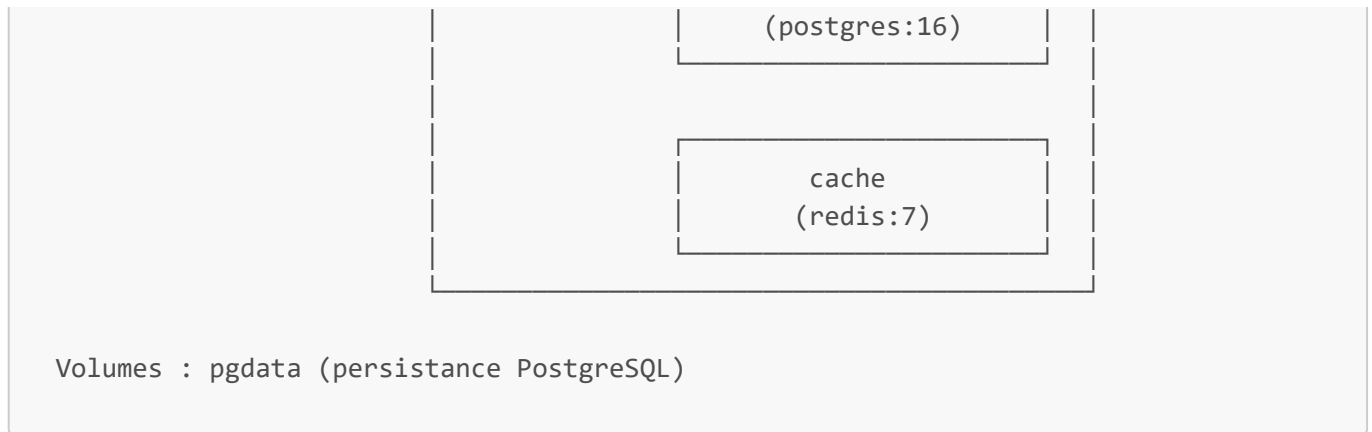
1. Architecture du stack

Le stack est composé de quatre services orchestrés par Docker Compose :

Service	Image / Build	Rôle
api	build: ./api	Backend FastAPI, expose les endpoints REST
db	postgres:16	Base de données relationnelle, persistance des données
cache	redis:7	Cache en mémoire, sert les lectures répétées
proxy	nginx:1.27	Point d'entrée HTTP, reverse proxy vers l'API

Schéma du stack et des dépendances





Ordre de démarrage garanti par les dépendances :

- **db** démarre en premier et doit être **healthy** (`pg_isready`)
- **cache** démarre librement
- **api** attend que **db** soit healthy avant de s'initialiser
- **proxy** attend que **api** soit démarré

2. Phase 1 — Squelette du stack

Capture 1 — Stack démarré

```
root@gitlab-gg000N-DAUVEL:/tmp/tp1-gg000N-dauvel# docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
tp1-gg000N-dauvel-api-1	tp1-gg000N-dauvel-api	"uvicorn app.main:ap..."	api	36 seconds ago	Up 30 seconds (healthy)	
tp1-gg000N-dauvel-cache-1	redis:7	"docker-entrypoint.s..."	cache	36 seconds ago	Up 35 seconds	6379/tcp
tp1-gg000N-dauvel-db-1	postgres:16	"docker-entrypoint.s..."	db	36 seconds ago	Up 35 seconds (healthy)	5432/tcp
tp1-gg000N-dauvel-proxy-1	nginx:1.27	"/docker-entrypoint..."	proxy	36 seconds ago	Up 24 seconds	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp

Le fichier `compose.yaml` déclare les quatre services, un réseau commun `tp-net` et un volume nommé `pgdata` pour la persistance de PostgreSQL :

```
services:
  api:
    build: ./api
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
    networks:
      - tp-net
    healthcheck:
      test: ["CMD-SHELL", "python3 -c \"import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')\" || exit 1"]
      interval: 5s
      retries: 5

  db:
    image: postgres:16
    env_file:
      - .env
    volumes:
```

```

- pgdata:/var/lib/postgresql/data
networks:
- tp-net
healthcheck:
test: ["CMD-SHELL", "pg_isready -U app"]
interval: 5s
retries: 5

cache:
image: redis:7
networks:
- tp-net

proxy:
image: nginx:1.27
depends_on:
api:
condition: service_healthy
volumes:
- ./proxy/nginx.conf:/etc/nginx/nginx.conf:ro
ports:
- "8081:80"
networks:
- tp-net

networks:
tp-net:

volumes:
pgdata:

```

Noms de services fixes : Les noms **api**, **db** et **cache** sont imposés par l'application — ils sont utilisés comme noms d'hôtes dans les variables **DATABASE_URL** et **REDIS_URL**. Les modifier casserait la résolution DNS interne.

3. Phase 2 — Healthchecks et dépendances

Capture 2 — Logs API attendant le healthcheck de la base

```

(*) up 6/6
✓ Image tpi-ggdon-dauvel-api Built 20.2s
✓ Network tpi-ggdon-dauvel-tp-net Created 0.1s
✓ Container tpi-ggdon-dauvel-cache-1 Created 0.4s
✓ Container tpi-ggdon-dauvel-db-1 Created 0.4s
✓ Container tpi-ggdon-dauvel-api-1 Created 0.1s
✓ Container tpi-ggdon-dauvel-proxy-1 Created 0.1s
Attaching to api-1, cache-1, db-1, proxy-1
Container tpi-ggdon-dauvel-db-1 Waiting
db-1 |
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
cache-1 | 11C 08 Jun 2026 12:17:22.190 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Being disabled, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
cache-1 | 11C 08 Jun 2026 12:17:22.190 * o080o0o0o0o0o Redis is starting o080o0o0o0o0o
cache-1 | 11C 08 Jun 2026 12:17:22.190 * Redis version=7.4.9, bits=64, commit=00000000, modified=0, pid=1, just started
cache-1 | 11C 08 Jun 2026 12:17:22.190 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
cache-1 | 11M 08 Jun 2026 12:17:22.190 * Increased maximum number of open files to 1024 (it was originally set to 1024).
cache-1 | 11M 08 Jun 2026 12:17:22.190 * monotonic clock: POSIX clock_gettime
cache-1 | 11M 08 Jun 2026 12:17:22.191 * Running mode=standalone, port=6379.
cache-1 | 11M 08 Jun 2026 12:17:22.191 * Server initialized
cache-1 | 11M 08 Jun 2026 12:17:22.191 * Ready to accept connections tcp
db-1 | 2026-06-08 12:17:22.211 UTC [1] LOG: starting PostgreSQL 16.14 (Debian 16.14-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2026-06-08 12:17:22.211 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-06-08 12:17:22.211 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2026-06-08 12:17:22.216 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-06-08 12:17:22.224 UTC [20] LOG: database system was shut down at 2026-06-08 12:13:15 UTC
db-1 | 2026-06-08 12:17:22.231 UTC [1] LOG: database system is ready to accept connections
Container tpi-ggdon-dauvel-db-1 Healthy
Container tpi-ggdon-dauvel-api-1 Waiting
api-1 | INFO: Started server process [1]
api-1 | INFO: Waiting for application startup.
api-1 | INFO: Application startup complete.
api-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
api-1 | INFO: 127.0.0.1:48850 - "GET /health HTTP/1.1" 200 OK
Container tpi-ggdon-dauvel-api-1 Healthy

```

Healthcheck de la base de données

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U app"]
    interval: 5s
    timeout: 5s
    retries: 5
```

`pg_isready` est l'outil officiel PostgreSQL pour vérifier que le serveur accepte les connexions. Docker exécute cette commande toutes les 5 secondes ; après 5 échecs consécutifs, le service est marqué **unhealthy**.

Healthcheck de l'API

```
api:
  healthcheck:
    test: ["CMD-SHELL", "python3 -c \"import urllib.request;
    urllib.request.urlopen('http://localhost:8000/health')\" || exit 1"]
    interval: 5s
    retries: 5
```

`curl` n'est pas installé dans l'image de base de l'API. On utilise donc le module `urllib` de la bibliothèque standard Python, disponible sans dépendance supplémentaire.

Dépendance conditionnelle

```
api:
  depends_on:
    db:
      condition: service_healthy
```

Sans condition: `service_healthy`, `depends_on` garantit uniquement l'**ordre de démarrage** des conteneurs, pas la disponibilité réelle du service. PostgreSQL peut mettre plusieurs secondes à accepter des connexions après le démarrage de son conteneur — sans healthcheck, l'API tenterait de se connecter trop tôt et échouerait.

4. Phase 3 — Configuration et secrets

Capture 3 — `.gitignore` et déclaration `env_file`

```
root@gitlab-gGODON-DAUVEL:/tmp/tp1-ggodon-dauvel# cat .gitignore && echo "----" && grep -A2 "env_file" compose.yaml
__pycache__/*
*.pyc
.env
.venv/
----
  env_file:
    - .env
  depends_on:
----
  env_file:
    - .env
  volumes:
```

Stratégie retenue

La configuration est externalisée dans un fichier `.env` (copié depuis `.env.example` fourni dans le dépôt). Ce fichier contient toutes les variables sensibles :

```
DATABASE_URL=postgresql+psycopg2://app:app@db:5432/app
REDIS_URL=redis://cache:6379/0
POSTGRES_USER=app
POSTGRES_PASSWORD=app
POSTGRES_DB=app
```

Le fichier `.env` est **exclu du dépôt** via `.gitignore` :

```
.env
*.env
```

Il est référencé dans `compose.yaml` via `env_file` :

```
services:
  api:
    env_file: .env
  db:
    env_file: .env
```

Ainsi, aucun secret n'apparaît ni dans les images Docker (pas de `ENV` dans le Dockerfile), ni dans le dépôt Git. Seul `.env.example` avec des valeurs fictives est versionné pour documenter les variables attendues.

5. Phase 4 — Overrides développement et production

Capture 4a — Profil développement

```
root@gitlab-ggdon-dauvel:/tmp/tp1-ggdon-dauvel# sudo docker compose down && sudo docker compose up -d && sudo docker compose ps
[+] down 5/5
✓ Container tp1-ggdon-dauvel-proxy-1 Removed 0.4s
✓ Container tp1-ggdon-dauvel-cache-1 Removed 0.4s
✓ Container tp1-ggdon-dauvel-api-1 Removed 0.6s
✓ Container tp1-ggdon-dauvel-db-1 Removed 0.3s
✓ Network tp1-ggdon-dauvel_tp-net Removed 0.2s
[+] up 5/5
✓ Network tp1-ggdon-dauvel_tp-net Created 0.6s
✓ Container tp1-ggdon-dauvel-db-1 Healthy 0.2s
✓ Container tp1-ggdon-dauvel-cache-1 Started 0.7s
✓ Container tp1-ggdon-dauvel-api-1 Healthy 11.8s
✓ Container tp1-ggdon-dauvel-proxy-1 Started 11.9s
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
tp1-ggdon-dauvel-api-1 tp1-ggdon-dauvel-api "uvicorn app.main:ap" api 12 seconds ago Up 6 seconds (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
tp1-ggdon-dauvel-cache-1 redis:7 "docker-entrypoint.s" cache 13 seconds ago Up 11 seconds 6379/tcp
tp1-ggdon-dauvel-db-1 postgres:16 "docker-entrypoint.s" db 13 seconds ago Up 11 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
tp1-ggdon-dauvel-proxy-1 nginx:1.27 "/docker-entrypoint..." proxy 12 seconds ago Up Less than a second 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
```

Capture 4b — Profil production

```
root@gitlab-ggdon-dauvel:/tmp/tp1-ggdon-dauvel# sudo docker compose down && sudo docker compose -f compose.yaml -f compose.prod.yaml up -d && sudo docker compose ps
[+] down 5/5
✓ Container tp1-ggdon-dauvel-cache-1 Removed 0.5s
✓ Container tp1-ggdon-dauvel-proxy-1 Removed 0.5s
✓ Container tp1-ggdon-dauvel-api-1 Removed 0.6s
✓ Container tp1-ggdon-dauvel-db-1 Removed 1.1s
✓ Network tp1-ggdon-dauvel_tp-net Removed 0.2s
[+] up 5/5
✓ Network tp1-ggdon-dauvel_tp-net Created 0.8s
✓ Container tp1-ggdon-dauvel-db-1 Healthy 5.9s
✓ Container tp1-ggdon-dauvel-cache-1 Started 8.5s
✓ Container tp1-ggdon-dauvel-api-1 Healthy 12.6s
✓ Container tp1-ggdon-dauvel-proxy-1 Started 12.5s
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
tp1-ggdon-dauvel-api-1 tp1-ggdon-dauvel-api "uvicorn app.main:ap" api 12 seconds ago Up 6 seconds (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
tp1-ggdon-dauvel-cache-1 redis:7 "docker-entrypoint.s" cache 12 seconds ago Up 12 seconds 6379/tcp
tp1-ggdon-dauvel-db-1 postgres:16 "docker-entrypoint.s" db 12 seconds ago Up 12 seconds (healthy) 5432/tcp
tp1-ggdon-dauvel-proxy-1 nginx:1.27 "/docker-entrypoint..." proxy 12 seconds ago Up Less than a second 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
```

Stratégie dev / prod par overrides

Plutôt que de maintenir deux fichiers `compose.yaml` complets (risque de désynchronisation), on utilise **un fichier de base** et **deux overrides** qui ne contiennent que ce qui diffère.

`compose.override.yaml` — Développement (chargé automatiquement)

```
services:
  api:
    volumes:
      - ./api/app:/app/app
    ports:
      - "8000:8000"

  db:
    ports:
      - "5432:5432"
```

- **Montage du code source** (`./api/app:/app/app`) : seul le répertoire applicatif est monté, les modifications sont immédiatement prises en compte sans rebuild.
- **Port 8000 exposé** : accès direct à l'API pour le debug, sans passer par le proxy.
- **Port 5432 exposé** : accès direct à PostgreSQL depuis l'hôte (ex. avec un client comme DBeaver ou psql) pour faciliter le debug en développement.

`compose.prod.yaml` — Production (chargé explicitement)

```
services:
  api:
    restart: always
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
```

```
    max-file: "3"
  deploy:
    resources:
      limits:
        cpus: "0.50"
        memory: 256M

db:
  restart: always
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
  deploy:
    resources:
      limits:
        cpus: "0.50"
        memory: 256M

cache:
  restart: always
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
  deploy:
    resources:
      limits:
        cpus: "0.25"
        memory: 128M

proxy:
  restart: always
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
  deploy:
    resources:
      limits:
        cpus: "0.25"
        memory: 64M
```

Lancement en production :

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

Lancement en développement :

```
docker compose up
```

6. Phase 5 — Limites de ressources et journalisation

Capture 5 — docker stats avec limites appliquées

```
root@gitlab-gODON-DAUVEL:/tmp/tp1-ggodon-dauvel# docker stats --no-stream
CONTAINER ID   NAME                                CPU %      MEM USAGE / LIMIT   MEM %      NET I/O      BLOCK I/O   PIDS
9cdd363a9c89   tp1-ggodon-dauvel-proxy-1         0.00%      2.102MiB / 64MiB     3.28%      796B / 126B   0B / 12.3kB  2
ed9783ac98e3   tp1-ggodon-dauvel-api-1           49.38%     51.91MiB / 256MiB    20.28%     3.62kB / 2.22kB 4.1kB / 0B   6
eb39fdf2f238   tp1-ggodon-dauvel-db-1            0.57%      22.63MiB / 256MiB    8.84%      3.92kB / 2.83kB 1.29MB / 618kB 7
acc8e27b0a82   tp1-ggodon-dauvel-cache-1         0.25%      3.172MiB / 128MiB    2.48%      1.91kB / 126B   0B / 0B      6
```

Limites de ressources (profil production)

Service	CPU max	Mémoire max
api	0.50 core	256 MB
db	0.50 core	256 MB
cache	0.25 core	128 MB
proxy	0.25 core	64 MB

Ces limites évitent qu'un service monopolise les ressources de l'hôte en cas de pic de charge ou de fuite mémoire.

Journalisation (profil production)

```
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
```

La rotation des logs est configurée pour **tous les services** (api, db, cache, proxy) et conserve au maximum **3 fichiers de 10 MB** par conteneur, soit 30 MB maximum par service. Sans rotation, les logs peuvent saturer le disque en production.

Ces réglages sont réservés au profil production (compose.prod.yaml) : en développement, les logs sont illimités pour faciliter le debug.

7. Phase 6 — Vérification de bout en bout

Capture 6 — Réponses API illustrant le passage base puis cache


```
root@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel# curl -X POST http://localhost:8081/items \
> -H "Content-Type: application/json" \
> -d '{"name": "test-tp2"}'
{"name":"test-tp2"}root@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel# curl http://localhost:8081/items
{"items":["test","test","test-tp2"],"source":"db"}root@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel# curl
{"items":["test","test","test-tp2"],"source":"cache"}root@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel#
```

Test de l'interconnexion complète

Création d'un item via le proxy :

```
curl -X POST http://localhost:8081/items \
-H "Content-Type: application/json" \
-d '{"name": "test-tp2"}'
```

Première lecture — source : base de données :

```
curl http://localhost:8081/items
# {"items": ["test-tp2"], "source": "db"}
```

Deuxième lecture — source : cache Redis :

```
curl http://localhost:8081/items
# {"items": ["test-tp2"], "source": "cache"}
```

Le champ **source** confirme que :

1. La première requête **GET /items** interroge PostgreSQL et stocke le résultat dans Redis (TTL 30s).
2. Les requêtes suivantes sont servies directement par Redis sans toucher la base.

L'interconnexion complète **Client → Proxy → API → PostgreSQL/Redis** fonctionne correctement via le réseau **tp-net**.

8. Phase 7 — Résilience

Capture 7 — Statuts illustrant la reprise après panne

```
ubuntu@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel$ sudo docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
tp1-ggdon-dauvel-api-1    tp1-ggdon-dauvel-api    "uvicorn app.main:ap..."    api        8 minutes ago    Up 8 minutes (healthy)    6379/tcp
tp1-ggdon-dauvel-cache-1    redis:7                "docker-entrypoint.s..."    cache      8 minutes ago    Up 8 minutes           5432/tcp
tp1-ggdon-dauvel-db-1      postgres:16            "docker-entrypoint.s..."    db         8 minutes ago    Up 3 seconds (health: starting)
tp1-ggdon-dauvel-proxy-1    nginx:1.27             "/docker-entrypoint..."    proxy      8 minutes ago    Up 7 minutes           0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
ubuntu@gitlab-gGODON-DAUVEL:/tmp/tp1-ggdon-dauvel$ sudo docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
tp1-ggdon-dauvel-api-1    tp1-ggdon-dauvel-api    "uvicorn app.main:ap..."    api        8 minutes ago    Up 8 minutes (healthy)    6379/tcp
tp1-ggdon-dauvel-cache-1    redis:7                "docker-entrypoint.s..."    cache      8 minutes ago    Up 8 minutes           5432/tcp
tp1-ggdon-dauvel-db-1      postgres:16            "docker-entrypoint.s..."    db         8 minutes ago    Up 22 seconds (healthy)
tp1-ggdon-dauvel-proxy-1    nginx:1.27             "/docker-entrypoint..."    proxy      8 minutes ago    Up 8 minutes           0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
```

Simulation de panne

```
docker compose restart db
docker compose ps
```

Comportement observé

1. **db** redémarre — son healthcheck passe en **health: starting**
2. L'API reste **healthy** pendant le redémarrage de la base grâce au `pool_pre_ping=True` de SQLAlchemy
3. **db** repasse en **healthy** après validation du `pg_isready`
4. Le stack est de nouveau pleinement opérationnel

Le healthcheck joue ici un rôle clé : sans lui, Docker ne saurait pas que **db** est de nouveau disponible, et la supervision (ou un orchestrateur comme Kubernetes) ne pourrait pas déclencher automatiquement la reprise.

9. Mini-questionnaire

1. À quoi sert un healthcheck, et que change `depends_on: condition: service_healthy` ?

Un **healthcheck** permet à Docker de sonder régulièrement l'état réel d'un service (et non simplement l'état de son conteneur). Un conteneur peut être **running** sans que le service à l'intérieur soit opérationnel — PostgreSQL met par exemple plusieurs secondes à accepter des connexions après le démarrage de son processus.

Sans `condition: service_healthy`, `depends_on` garantit uniquement que le conteneur dépendant **démarre après** le conteneur cible — pas qu'il soit prêt à recevoir des connexions. Avec `condition: service_healthy`, Docker attend que le healthcheck du service cible retourne un succès avant de démarrer le service dépendant, éliminant les erreurs de connexion au démarrage.

2. Comment Compose isole-t-il les services en réseau par défaut ?

Docker Compose crée automatiquement un **réseau bridge dédié** au projet (nommé `<projet>_default` par défaut, ou le réseau déclaré explicitement). Tous les services du stack y sont connectés et peuvent se joindre par leur **nom de service** grâce au DNS interne de Docker.

Les services ne sont **pas accessibles depuis l'extérieur** sauf si un `ports` est explicitement déclaré. Dans notre stack, seul le proxy expose le port 8081 — `api`, `db` et `cache` restent isolés sur `tp-net` et ne sont joignables que depuis les autres conteneurs du même réseau.

3. Pourquoi structurer dev et prod par des overrides plutôt qu'en deux fichiers complets ?

Deux fichiers complets entraînent inévitablement de la **duplication** : toute modification de la configuration commune (ajout d'un service, changement d'image) doit être répercutée dans les deux fichiers, ce qui crée des risques de désynchronisation et augmente la charge de maintenance.

Les **overrides** appliquent le principe DRY (Don't Repeat Yourself) : `compose.yaml` contient la configuration commune, et chaque override ne contient **que ce qui diffère**. Le merge est géré automatiquement par

Compose, garantissant que les deux environnements partagent toujours la même base.

4. Où placer un secret pour qu'il ne figure ni dans l'image ni dans le dépôt ?

Dans un fichier `.env` présent uniquement sur la machine d'exécution (jamais commité — exclu via `.gitignore`), référencé via `env_file` dans `compose.yaml`. Les variables sont injectées dans les conteneurs au démarrage comme variables d'environnement, sans jamais être écrites dans l'image.

En production réelle, on utiliserait des solutions dédiées comme **Docker Secrets** (Swarm), **Vault** (HashiCorp), ou les secrets natifs du cloud provider — mais le fichier `.env` local est la solution adaptée au contexte de ce TP.

5. Que deviennent les données si aucun volume nommé n'est déclaré pour PostgreSQL ?

Les données sont stockées dans la **couche inscriptible du conteneur** (layer writable). Elles sont donc **perdues dès que le conteneur est supprimé** (`docker compose down` sans `-v` les conserve, mais `docker compose down -v` ou `docker rm` les efface définitivement).

Sans volume nommé, chaque `docker compose up` recrée un PostgreSQL vide. Le volume nommé `pgdata` découple le cycle de vie des données de celui du conteneur — les données survivent aux suppressions et créations de conteneurs.
